



WASMハイパーバイザをSDDで開発してみた

fireball開発で得たノウハウ

発表者: kmt-t

SPECIFICATION_DRIVEN_DEVELOPMENT // SYSTEM_INIT



自己紹介



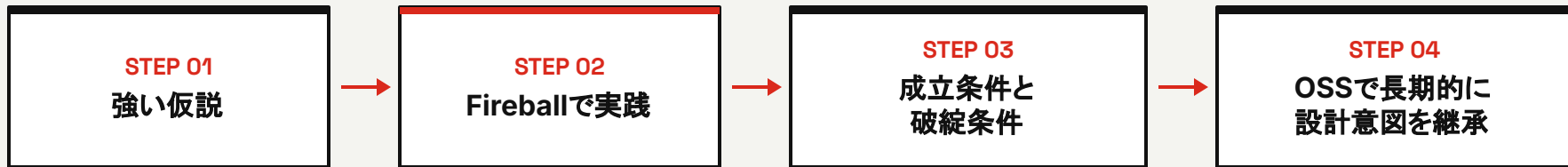
発表者:kmt-t

- センサーメーカー勤務
- 組み込みプログラマとして、ハードウェアに近い領域のソフトウェア開発に従事
- リソース制約の厳しい環境でのコーディングが中心
- 最近の興味は家事と節約
- AI支援開発の進展で、過密な開発スケジュールで少し疲労気味





- 強い仮説を、実際の開発を通じて検証する
- **fireball** にSDDを適用し、正常に機能する条件と破綻する条件を明らかにする
- 完全な仕様書が、将来のフォークや別環境への移植で設計意図を継承する土台になるか確認する



本報告の目的

SDD一般の抽象論ではなく、具体的なプロジェクト **fireball** の実践から有効性と限界を探る。

SDD導入における4つの主な懸念点



01 // SPEED

実装速度の低下

詳細な仕様作成に追われ、実装が遅れるのではないか

02 // ACCURACY

誤った仕様の固定化

初期理解が間違っていた場合、誤った仕様を強固に固めてしまうのではないか

03 // MAINTENANCE

メンテナンスコストの増大

仕様変更のたびにコードと大量のドキュメントを同時に直す必要があるのではないか

04 // VERIFICATION

実機検証のギャップ

ハードウェア上で動かさないと分からない問題を、仕様書だけで扱えるのか



本セッションの対象とフォーカス

01. 想定する対象

**SDDを試したいソフトウェア設計者
・実装者**

AI支援開発の文脈で、仕様駆動開発（SDD）を取り入れたい方を対象とします。

ソフトウェアアーキテクチャ、仕様策定、テスト手法の基本知識を共有していることを前提とします。

02. 前提としない知識

深層の低レイヤ専門知識は不要

WASMの内部仕様やJITコンパイラの詳細な設計知識は求められません。

形式検証に関する高度な専門理論を知らなくても、セッションの内容を十分に理解していただけます。

03. セッションの重点

現場のリアルな問題と解決ノウハウ

Fireballの開発現場で実際に起きた課題、具体的な解決策、得られたノウハウを扱います。

固有技術はSDDの有効性と課題を示すためのケーススタディとして活用します。

極限のハードウェア制約



32 KB

RAM CAPACITY

96 KB

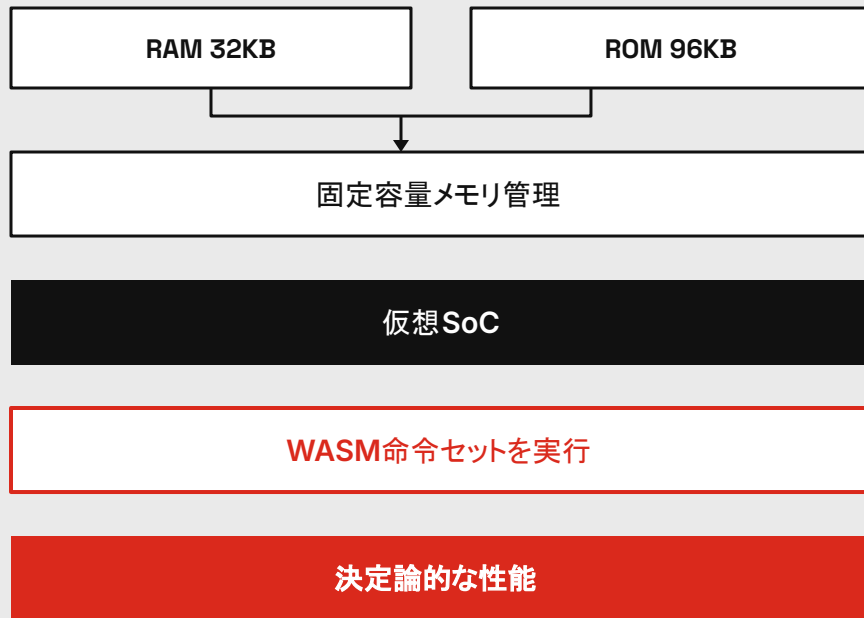
ROM CAPACITY

- 動的メモリ割り当てに依存しない固定容量メモリ管理
- 決定論的な実行性能を重視
- WASM命令セットをネイティブに解釈・実行する仮想 SoC
- RAM 32KB、ROM 96KBという制約に特化

性能目標

特定の組み込みユースケースに極限特化し、WAMRを上回る実行性能を目指す。

ARCHITECTURAL EXECUTION FLOW

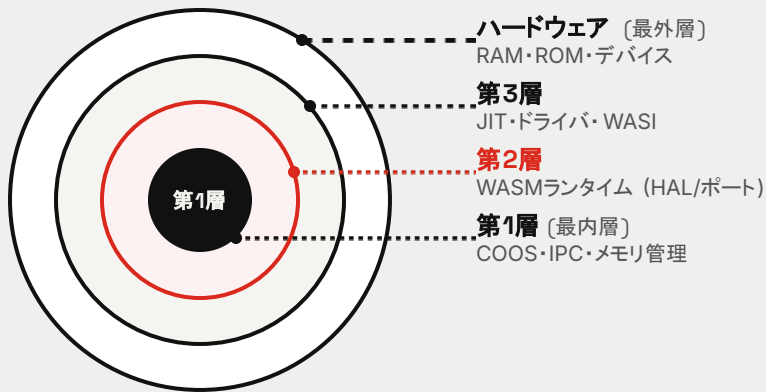




システム設計とレイヤー依存関係

概要: 役割を分け、依存は外側から内側へ。ハードウェアは外側の具体物で、内部ではHAL／ポートとして扱う。

[DEPENDENCY: OUTER → INNER]



階層仕様 & 依存ルール

- 第1層: ハードウェアを直接見ない
- 第3層: ドライバがハードウェアへ直接アクセスする
- 第2層: HAL／ポートとして抽象化して扱う
- 依存の向き: 常の外側から内側へ単方向に限定

現在の状況とこれからの展開

[CURRENT STATUS]

Pythonシミュレータ

フルセットが動作中 (実装率 100%)

全体アーキテクチャおよびプロトタイプの検証を完了。

[NEXT STEP]

実機向け C++ 移植 & 動作確認

設計に沿って実機ターゲットへ移植を推進。

TARGET SPECS: RAM 32KB / ROM 96KB

AI開発の罠: 動くコードが隠す設計負債



STATUS // シミュレータ段階

Pythonシミュレータ段階では、フルセットの機能が動作する状態に到達

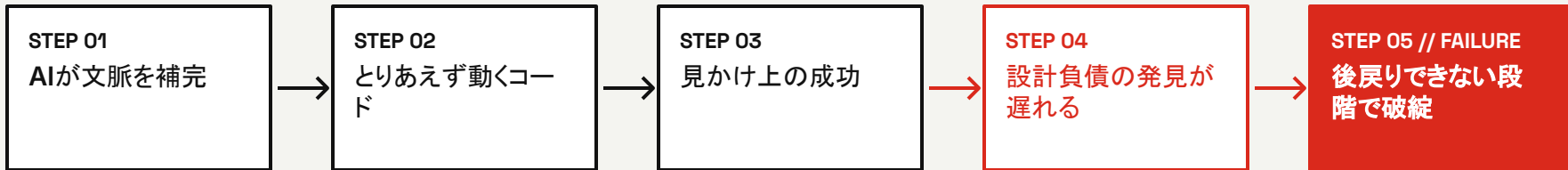
REALITY // 設計完成度

ただし、アーキテクチャや仕様といった設計の完成度は、まだ 50% 程度

CRITERIA // 評価の誤解

「テストが通って動いたから OK」だけでは、設計品質を判断できない

PROCESS FLOW // AIアシスト開発で生じる見かけ上の成功と破綻



WARNING // AI開発最大の罠

設計が矛盾を抱えていても、AIが強引に辻褃を合わせてコードを生成できてしまう。動いてしまうことが、本質的な設計負債を隠す。

メモリ制約下のデータ構造と検索アルゴリズム



- RAM 32KBの制約下では、静的なデータを ROMに置く必要がある。一方、一般的な C++標準ライブラリは ROM上のデータ操作や静的配置を十分に想定していない。
- そのため、極小 RAM環境に特化した独自コンテナと、高速かつ決定論的な 3段階検索アルゴリズムを採用する。

高速・決定論的 3段階検索パイプライン

STEP 01

キャッシュ検索

計算量: $O(1)$

16エントリ的高速キャッシュを参照。頻出データへの即時アクセスを実現。

STEP 02

基数表

計算量: $O(1)$

基数インデックスを用いて最終探索対象の検索範囲を大幅に絞り込む。

STEP 03

二分探索

計算量: $O(\log N)$

絞り込まれた限定領域に対して決定論的な最終検索を実施し確定。



CAUSALITY FLOW // HARDWARE TO SPEC

[01] HARDWARE LIMITS

RAM 32KB / ROM 96KB

標準ライブラリの想定範囲を逸脱

標準的な代替手段が使えない

独自アルゴリズムとデータ構造が必要

設計複雑度が上がる

[CONCLUSION]

詳細仕様を先に定義する必要がある

CRITICAL WARNING // AI CODING RISKS

詳細設計なきAIコーディングへの警告

■ 実装選択の無秩序化と拡散

詳細設計を省略してAIにコーディングを委ねると、AIが選択できる実装の幅が広がりすぎる。

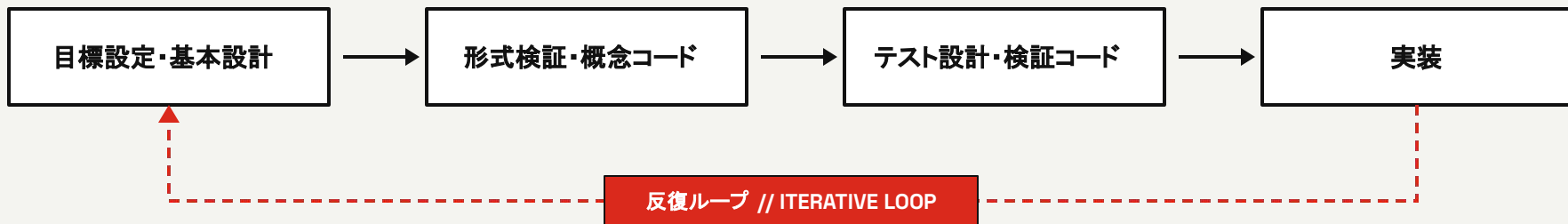
その結果、システム全体で整合性のない、無秩序で予測不可能な実装が生成される。

CORE PRINCIPLE // ガードレール

詳細仕様は、
AIの暴走を防ぐガードレールになる。

// SPECIFICATION STOPS UNPREDICTABLE CODE

古典的な設計先行型アプローチ



破綻の根本原因

検証ステップを刻まないと実装へ進めない一方、ステップを重ねるほど源流の要求との対応がぼやけ、各段階の仕様解像度が落ちていった。その結果、検証の拠り所となる仕様・設計の品質が不足し、LLMを活用した高速な開発サイクルではプロセス全体が機能不全に陥った。

仕様駆動開発の課題: 文書肥大化と AIチェックの形骸化



01. 文書量の増加とプロセスの形骸化

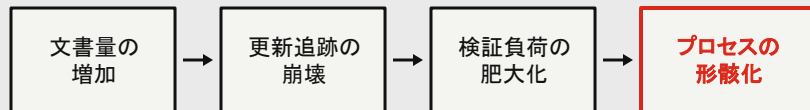
- 仕様変更に従従できず、文書間の不一致や記述の不整合が多発
- 文書量が増え、更新箇所や影響範囲の特定・追跡が難しくなる
- 整合性の維持・検証そのものが過度な負担になる

02. AIの形骸チェックと「動くコード」の罠

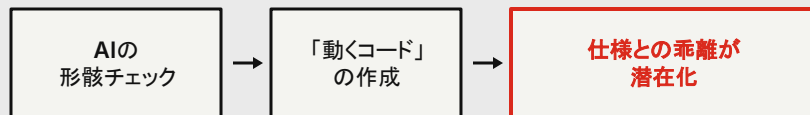
- AIが「確認済み」としてチェック作業を省略し、検証漏れが発生
- 仕様に反していてもコードが動くため、本質的な乖離が潜在化
- 文書量を増やすだけでは、正しさは保証されない

PROCESS DECAY & AI TRAP FLOW

[FLOW A] 文書肥大化による崩壊シナリオ



[FLOW B] AIチェックと動くコードの罠



CRITICAL_RISK: 手動/AIを問わず、目視確認に依存した検証プロセスは文書量の増加により必ず崩壊する。

TAKEAWAY // 本質的な教訓

ドキュメントの量を増やすだけでなく、正しさを自動かつ強制的に確認する仕組みが必要。

組み込みC++への移植を想定した 8つの監査観点



01 // 仕様一致

仕様書、状態機械、GOTCHAと実装に乖離がない

02 // 型安全性

すべての変数・引数に具体的な型を付け、`typing.Any`を排除

03 // 固定容量

動的アロケーションを伴う`dict`、`set`、伸縮リストを禁止

04 // 計算量・決定論

ホットパスの線形探索や実行時アロケーションを排除

05 // ROM/RAM配置

不変データをROMに置き、RAM使用量を最小化

06 // メモリ効率

冗長なバッファ構造や`__dict__`を排除

07 // デッドコード

未使用関数やデバッグ分岐を削除

08 // Tier境界

責務分割と依存方向を仕様・実装で一致させる

4層設計チェーンの層間一貫性



仕様

形式検証

概念コード

テスト

仕様 ⇔ 形式検証

状態、遷移、安全性、活性、変異検査が的一对で対応するか

形式検証 ⇔ 概念コード

証明した状態遷移や制約がコードで再現されているか

概念コード ⇔ テスト

境界条件、GOTCHA、状態遷移のエッジケースを網羅するか

ドキュメント ⇔ 実態

解説、図、疑似コード、API定義にズレがないか

追加検証 (ADDITIONAL VERIFICATION)

- 内側の層が外側の層を参照していないか確認
依存関係の方向性が単一方向(外側→ 内側)に保たれているかを厳密に検証
- キーワード、検証証跡、逆リンクの整合性を確認
トレーサビリティを確保し、全レイヤー間に矛盾や追跡不能な変更がないかを監査



仕様と実装の乖離を機械的に検出する 7つのゲート

[INPUT SOURCE] 仕様 (Specification) & 実装 (Implementation) & 証跡 (Evidence)



01 FORMAT

構文検証

- ・リンク切れ
- ・アンカー参照
- ・Mermaid

[Syntax Gate]

02 TRACE

追跡性

- ・未定義語
- ・未参照要求
- ・孤立要素

[Ref Gate]

03 HIERARCHY

階層構造

- ・Tier逆流
- ・依存方向違反
- ・境界越境

[Arch Gate]

04 FORMAL

形式検証

- ・契約欠落
- ・到達不能状態
- ・空虚な証明

[Logic Gate]

05 EVIDENCE

証跡検証

- ・テスト証跡
- ・ベンチマーク
- ・実行ログ欠落

[Proof Gate]

06 OBLIGATE

監査・責務

- ・高リスクタグ
- ・監査漏れ
- ・規約違反

[Audit Gate]

07 CONSIST

整合性

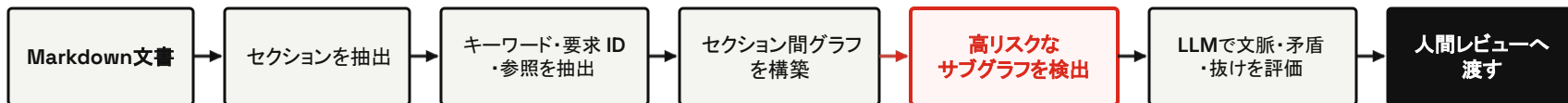
- ・重複定義
- ・定数値不一致
- ・型矛盾

[Sync Gate]

PIPELINE 01 // 仕様ドキュメントのグラフ抽出と高リスク検出

セクションとキーワードから検証対象を組み立てる:

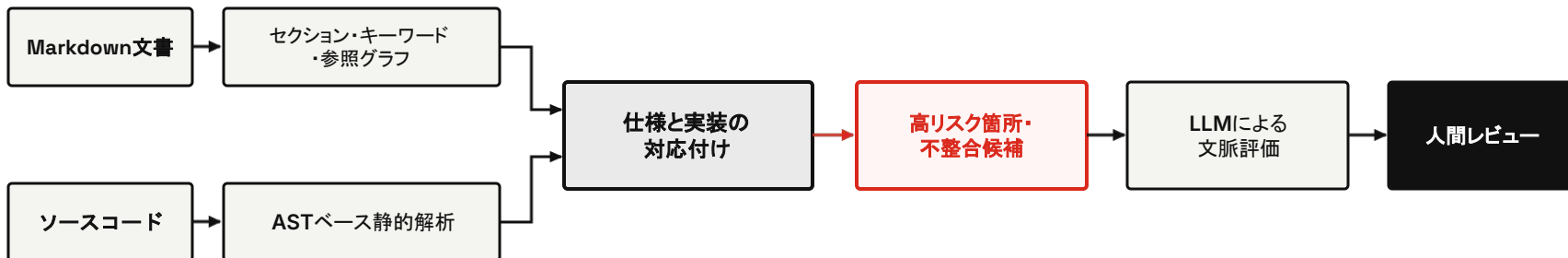
- ・Markdownの文書を、セクションをノード、キーワード・要求ID・参照関係をエッジとするグラフとして扱う。
- ・関係の強い範囲をサブグラフとして取り出し、リスクの高い箇所を優先して評価する。



PIPELINE 02 // ソースコードAST解析と仕様・実装の相互照合

ソースコード側はASTベースで静的解析する:

- ・ソースコードをASTとして解析し、文書グラフとの対応付けから不整合を絞り込む。

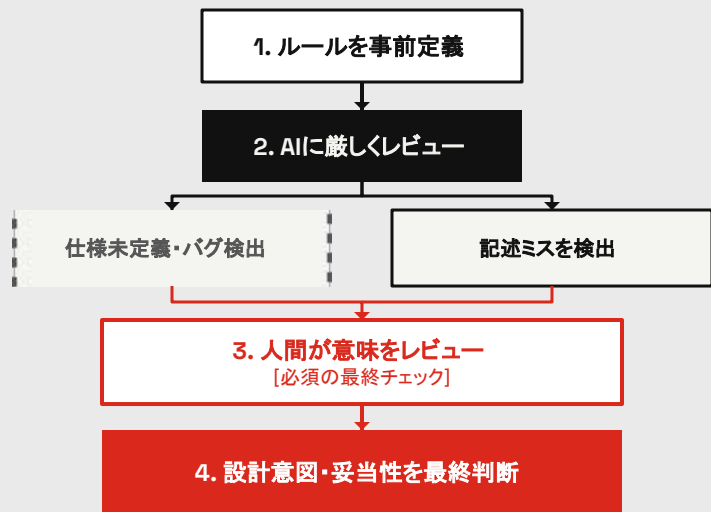


自動チェックの限界と人間の監査



ルールを決めてAIに厳しくレビューさせると、仕様未定義や仕様バグがかなり見つかる。検証は人間の記述ミスも見つけるが、人間レビューなしでは気づけない。

REVIEW WORKFLOW // 人間とAIの協調



人間が担う監査項目

■ 表記ゆれと同義語

文脈上の使い分けを判断

■ 論理的一貫性

主張、解説、図、疑似コード、根拠データを照合

■ 垂直一貫性

数値、ハードウェア制約、ADRと最新仕様を照合

■ 意味論の正当性

テスト証跡、自然言語の明確さ、不要な参照を確認

クリーンアーキテクチャの原則を仕様書へ適用



SPECIFICATION PRINCIPLES // 基本原則

- 外側の仕様は内側に依存してよい
- 内側の安定コアは外側を一切参照しない

CRITICAL RULE // 重大規制

内側から外側への参照は、単なる誤記ではな**重大な不具合**

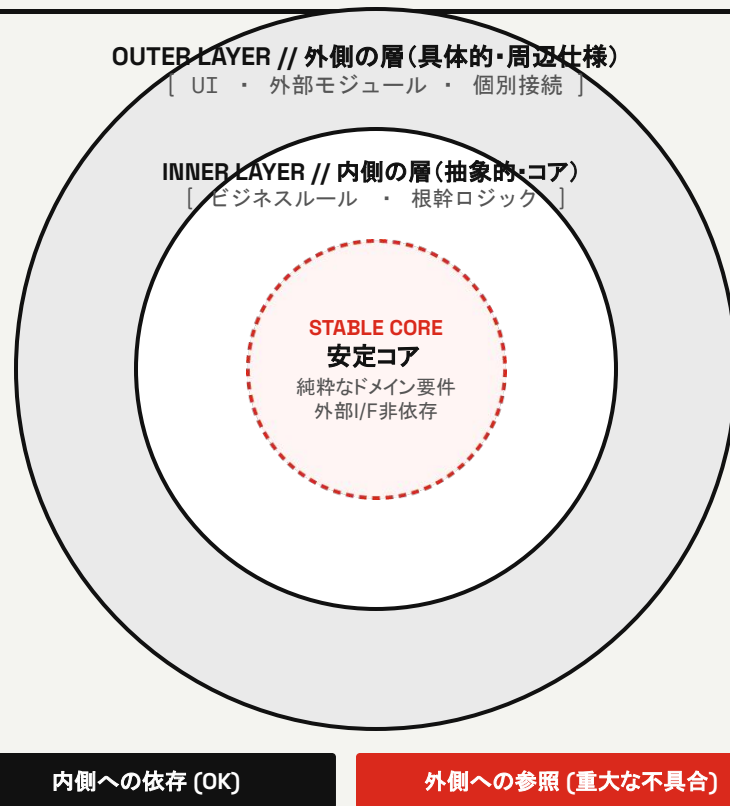
MEASURES & EFFECTS // 対策と効果

01. 内側仕様の完全自立

内側の仕様書から外側の概念・記述への参照を削除し、内側を自立させる

02. 変更影響の最小化

仕様変更時の影響範囲がアーキテクチャから明確になり、予測可能で最小限になる





観点の固定とAIサブエージェント協調

01. 観点の事前固定

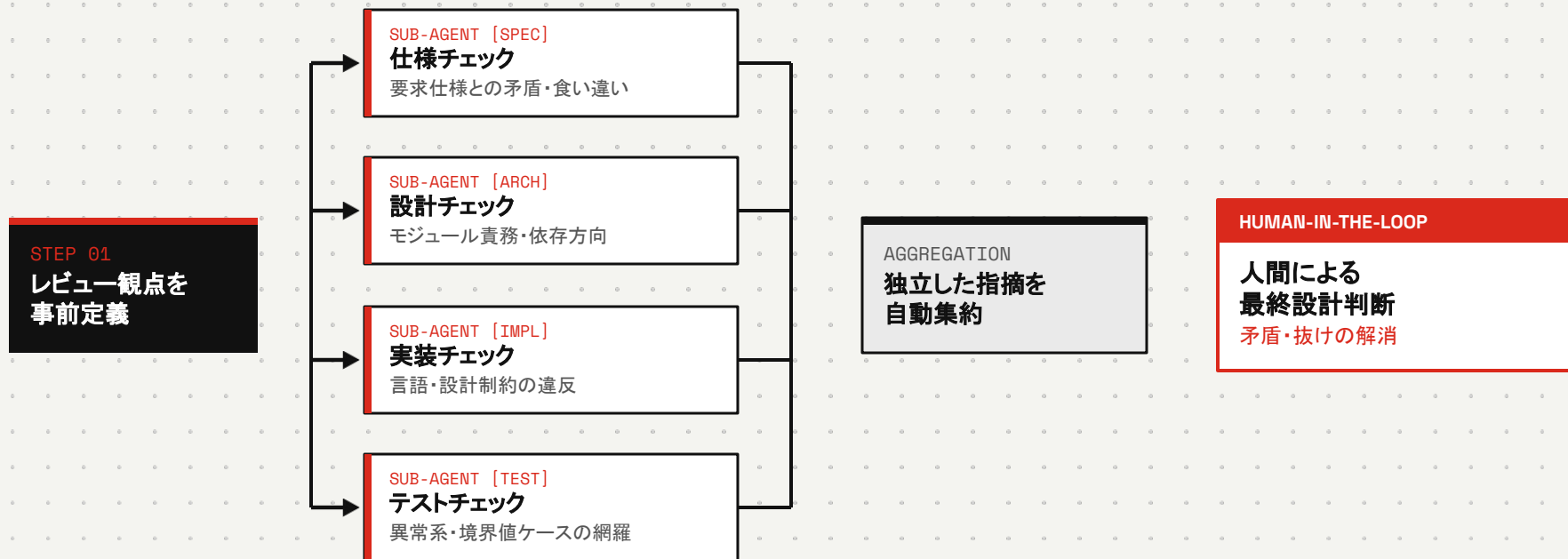
コード記述前に「何を見ればよいか」の評価軸を定義

02. 専門化サブエージェント

巨大プロンプトを回避し、領域特化型 AI群に分散処理

03. 人間による最終判断

集約された矛盾・抜けを人間が評価し意志決定

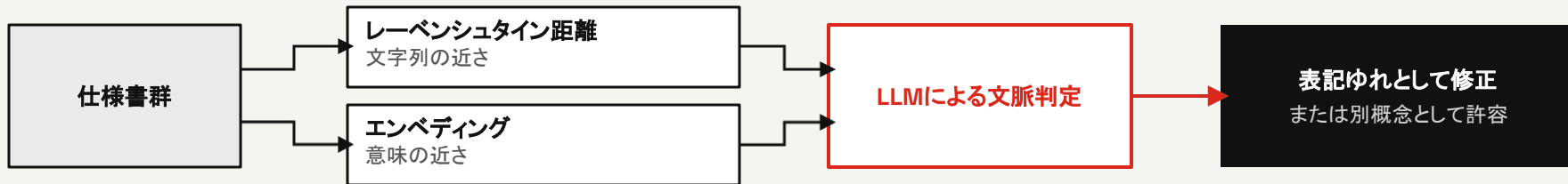


検証と精度の自動化処理



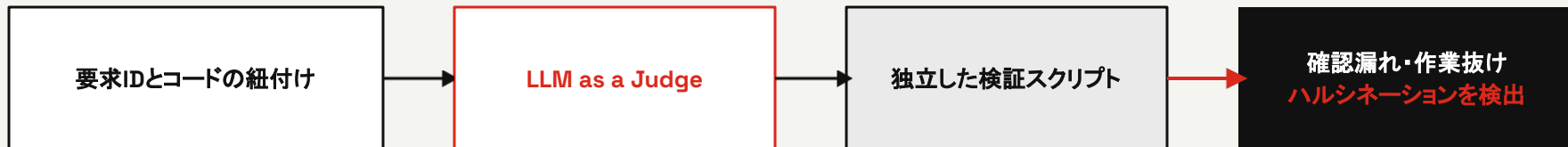
表記ゆれと同義語の自動抽出

レーベンシュタイン距離とエンベディングを用いて候補を抽出し、LLMによる文脈判定で修正または許容を決定する。

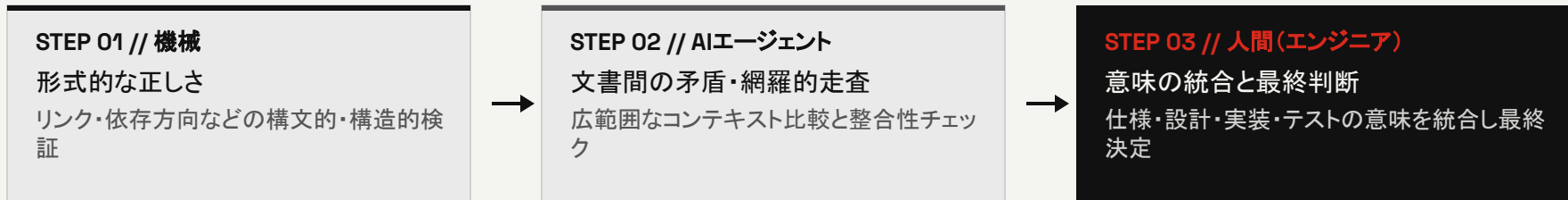


トレーサビリティと作業抜けのチェック

要求IDとコードの紐付け、LLM as a Judgeを組み合わせた独立した検証スクリプトで、確認漏れ・ハルシネーション・作業抜けを検出する。



人間と機械の役割分担



鉄則

自動化が代替するのは確認作業の根気であり、エンジニアの思考ではない。

人間への要求

- **技術理解と自律設計**：技術を理解し、自分の頭で設計できること。
- **最終責任の所在**：AIやツールを使ったSDD(仕様駆動開発)であっても、最後の責任は人間が負う。



仕様駆動開発に関する 4つの懸念と検証

01. 実装が遅くならないか？

初期の仕様作成コストは増えるが、仕様不在によるAIの暴走や設計破綻を防ぐため、

→ 中長期の総開発時間は削減できる

02. 誤った仕様を固めないか？

実行結果から得た知見を仕様へ逆流させる仕組みを作れば、

→ 仕様は進化する

03. 変更コストが増えないか？

仕様書の依存関係を整理すれば、

→ 影響範囲を絞り込める

04. 動かさないと分からない問題は？

シミュレータや実機確認と仕様化を素早く往復する

→ スパイラル開発で扱う

SDDを成功させる 4つの開発方針



01

内側の層を安定させる

コア仕様から外側の実装依存を排除する

02

レビュー観点を事前定義する

何を正しいとするかをコードの前に決める

03

依存関係に沿って追跡する

要求、仕様、設計、実装、テストを依存方向に沿って紐付ける

04

自動チェックと人間判断を組み合わせる

自動化は漏れを減らし、人間は設計意図と最終判断を担う



方針を実開発で機能させる 6ステップ

01 依存方向定義

アーキテクチャ図で文書とコードの依存方向を決める

02 観点定義

設計レベルの評価・レビュー観点を定める

03 専門レビュー

観点別のサブエージェントで独立レビューする

🔄 CONTINUOUS_FEEDBACK_LOOP

06 フィードバック

テストやシミュレータの結果を仕様と設計へ反映し、1へ戻る

05 人間による読み比べ

人間が仕様からテストまでを読み比べ、意味的な整合性を確認する

04 機械チェック

リンク、用語、数値、証跡を自動チェックする

運用のヒント

規模、チームのリソース、許容できるリスクに応じて、循環モデルを調整して運用する。

ディスカッション & 今後の課題



課題1: 設計の完成度向上

極限環境の組み込み WASMハイパーバイザにおいて、SDDを適用する境界や範囲は現在の設定で適切か。

課題2: 仕様と実装のズレの最小化

4層の設計チェーンなど、現在のレビュー観点にまだ抜け漏れがないか。

課題3: 検証プロセスの軽量化

仕様の厳密な整合性を保ちながら、検証を軽量化し開発スピードを上げる方法はないか。

課題4: AIの作業抜け対策

サブエージェントの分担と、人間が必ず理解し責任を持つ範囲をどう定義するか。